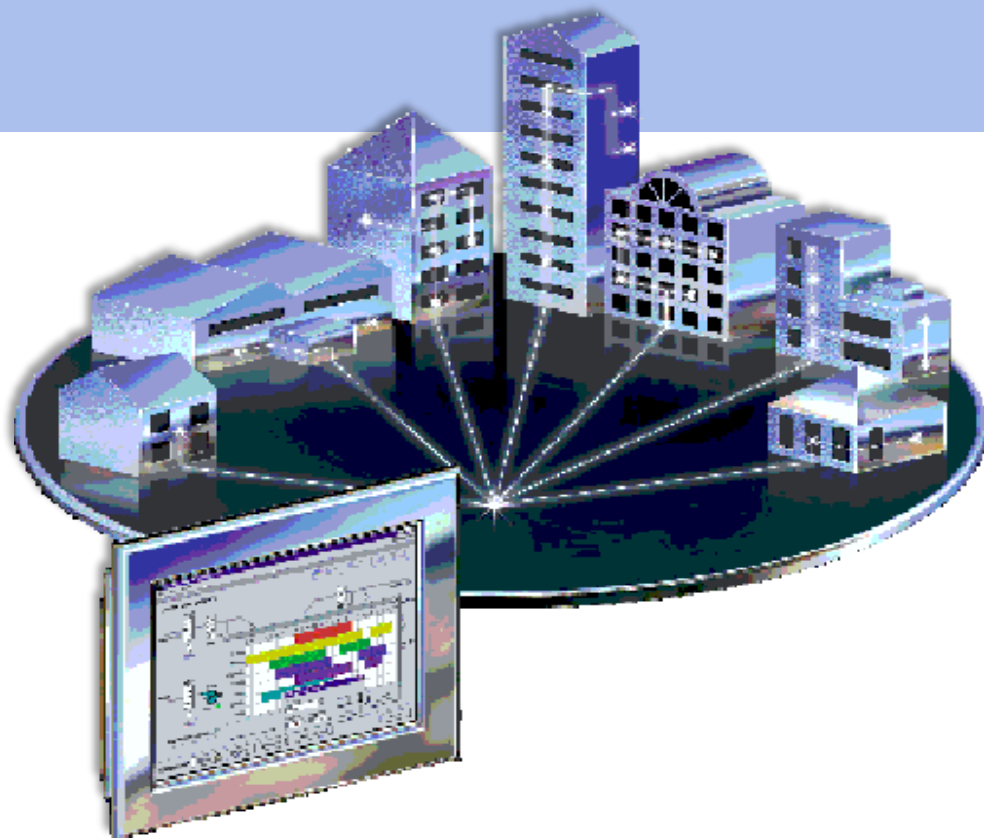


INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Práctica 1:

MULTIPROCESAMIENTO



SISTEMAS DISTRIBUIDOS

Profesor || Chadwick Carreto Arellano

Grupo || 7CM1

Alumno || Areli Alejandra Guevara Badillo

Febrero 13, 2026

ANTECEDENTES

Evolución de la Computación: Secuencial vs. Paralelo

Históricamente, los programas se ejecutaban de forma secuencial. Esto significa que una instrucción debía finalizar por completo antes de que la siguiente pudiera comenzar. Bajo este modelo, si una tarea era muy pesada (como procesar miles de datos), el procesador se convertía en un "cuello de botella", dejando otros recursos del sistema inactivos.

Con la llegada de los procesadores de múltiples núcleos (Multi-core), surgió la necesidad de la computación paralela, que permite que múltiples partes de un programa se ejecuten simultáneamente en diferentes unidades de procesamiento. El multiprocesamiento permite que varias unidades centrales de procesamiento realicen operaciones en paralelo, compartiendo los mismos recursos principales del sistema como la memoria, el bus de datos y los dispositivos periféricos. Esto significa que múltiples instrucciones pueden ejecutarse de forma real y simultánea, lo que incrementa el rendimiento y optimiza el aprovechamiento del hardware disponible.

Diferencia entre Proceso e Hilo

Es crucial no confundirlos para entender el multiprocesamiento:

- **Proceso:** Es un programa en ejecución que tiene su propio espacio de memoria aislado. Si un proceso falla, no afecta a los demás.
- **Hilo:** Vive dentro de un proceso. Comparten recursos (memoria, archivos abiertos). Son más rápidos de crear y destruir que los procesos.

Beneficios del Multiprocesamiento

- **Capacidad de respuesta:** La aplicación no se "congela" mientras realiza una tarea pesada en segundo plano.
- **Economía:** Es más barato crear hilos que procesos completos.
- **Escalabilidad:** El programa aprovecha automáticamente si el usuario tiene un procesador con más núcleos.

Riesgos y Desafíos del Multiprocesamiento

- **Condiciones de Carrera (Race Conditions):** Ocurre cuando dos o más hilos intentan acceder y modificar un mismo dato al mismo tiempo.
- **Bloqueos Mutuos (Deadlocks):** Sucede cuando el Hilo A está esperando un recurso que tiene el Hilo B, y el Hilo B está esperando un recurso que tiene el Hilo A. Ambos se quedan esperando indefinidamente y el programa se "traba".
- **Mecanismos de Sincronización.**

PLANTEAMIENTO DEL PROBLEMA

La problemática que se plantea en esta practica es la ineficiencia que surge al intentar procesar grandes volúmenes de imágenes (procesamiento masivo) de forma secuencial. En una aplicación tradicional de un solo hilo, cada imagen debe esperar a que la anterior termine por completo para ser procesada. Esto genera un principal y gran inconveniente: tiempos de espera muy largos.

PROPUESTA DE SOLUCIÓN

Para resolver esta ineficiencia, la propuesta consiste en implementar un modelo de **multiprocesamiento a través de múltiples hilos** para la ejecución de tareas en paralelo. Esta estrategia permite atacar el problema desde tres ángulos:

Estrategia de Implementación:

- **Segmentación de Carga:** Se propone dividir el lote total de trabajo (las imágenes) en N subconjuntos iguales, donde N idealmente corresponde al número de hilos o núcleos lógicos disponibles en el procesador.
- **Ejecución Concurrente:** Se instanciarán múltiples hilos de ejecución independientes. Cada hilo recibirá un segmento de los datos y realizará la transformación (filtro de imagen) de manera simultánea a los demás.
- **Gestión de Hilos:** Se utilizarán mecanismos de control para asegurar que el programa principal espere la finalización de todos los hilos antes de entregar el resultado final, garantizando la integridad de los datos.

DESARROLLO E IMPLEMENTACIÓN

Para resolver la problemática de la ineficiencia en el procesamiento masivo de imágenes, se desarrolló una aplicación en Python que implementa un modelo de multiprocesamiento mediante hilos, permitiendo ejecutar múltiples tareas de forma concurrente. El objetivo principal de la implementación es reducir significativamente los tiempos de espera al distribuir la carga de trabajo entre varios hilos de ejecución.

La aplicación fue construida utilizando la librería *Tkinter* para la interfaz gráfica, *Pillow (PIL)* para el procesamiento de imágenes y *threading* junto con *multiprocessing* para la gestión de concurrencia.

Arquitectura general del sistema

El sistema se compone de tres elementos principales:

Práctica 1: Multiprocesamiento

1. **Interfaz gráfica:** Permite al usuario seleccionar una carpeta con imágenes y visualizar el progreso del procesamiento.
2. **Módulo de segmentación de carga:** Divide el conjunto total de imágenes en subconjuntos.
3. **Módulo de ejecución concurrente:** Lanza múltiples hilos que procesan los subconjuntos en paralelo.

Esta arquitectura permite desacoplar la interacción con el usuario del procesamiento pesado, manteniendo la aplicación responsiva durante la ejecución.

Segmentación de carga de trabajo

Siguiendo la estrategia propuesta, el lote total de imágenes se divide dinámicamente en N subconjuntos, donde N corresponde al número de núcleos lógicos disponibles en el sistema:

```
1 self.hilos_activos = []
2 num_hilos = multiprocessing.cpu_count()
3 paso = len(archivos) // num_hilos
```

De esta forma, cada hilo recibe una fracción del total de imágenes, logrando una distribución equitativa de la carga. Esta segmentación permite que el trabajo se realice de manera balanceada y evita que un solo hilo concentre todo el procesamiento.

Ejecución concurrente mediante hilos

Una vez segmentados los datos, se instancian múltiples hilos de ejecución utilizando la librería `threading`. Cada hilo ejecuta de forma independiente la misma función de procesamiento (`procesarImg`), pero sobre un subconjunto distinto de imágenes.

```
1 for i in range(num_hilos):
2     inicio = i * paso
3     fin = None if i == num_hilos - 1 else (i + 1) * paso
4     lote = archivos[inicio:fin]
5
6     t = threading.Thread(target=self.procesarImg, args=(lote, origen, destino, f"hiloTrabajador{i+1}"), daemon=True)
7     self.hilos_activos.append(t)
8     t.start()
```

Este enfoque implementa un modelo de paralelismo por datos, donde múltiples hilos realizan la misma operación (conversión a escala de grises) sobre diferentes datos al mismo tiempo. Gracias a esto, varias imágenes son procesadas simultáneamente, reduciendo el tiempo total de ejecución.

Práctica 1: Multiprocesamiento

Función de procesamiento pesado

La función `procesar_img` representa la tarea intensiva del sistema. Cada hilo recorre su lista de imágenes asignadas y aplica una transformación de escala de grises utilizando la librería *Pillow*.

```
1 with Image.open(ruta_in) as img:
2     img_gris = ImageOps.grayscale(img)
3     img_gris.save(ruta_out)
```

Sincronización y control de concurrencia

Debido a que múltiples hilos comparten recursos comunes (como la variable de progreso), fue necesario implementar mecanismos de sincronización para evitar inconsistencias.

Para ello se utilizó un objeto `Lock`. Este bloqueo garantiza que solo un hilo a la vez pueda modificar el estado de la barra de progreso:

```
1 self.lock = threading.Lock()
2 ...
3 with self.lock:
4     actual = self.progreso_var.get()
5     self.root.after(0, self.progreso_var.set, actual + (100 / self.total_fotos))
```

Esto previene condiciones de carrera (*race conditions*) y asegura que la información mostrada al usuario sea correcta.

Gestión y finalización de hilos

Para cumplir con la estrategia de gestión de hilos, el sistema implementa un proceso de monitoreo que espera a que todos los hilos terminen su ejecución antes de notificar el resultado final.

```
1 def monitor_finalizacion(self):
2     for t in self.hilos_activos:
3         t.join()
4     self.root.after(0, self.notificar_termino)
```

Este mecanismo garantiza que el programa principal no finalice prematuramente y que todas las imágenes hayan sido procesadas correctamente antes de mostrar los resultados.

Práctica 1: Multiprocesamiento

Medición de desempeño

El tiempo total de ejecución se calcula registrando el instante de inicio y fin del procesamiento:

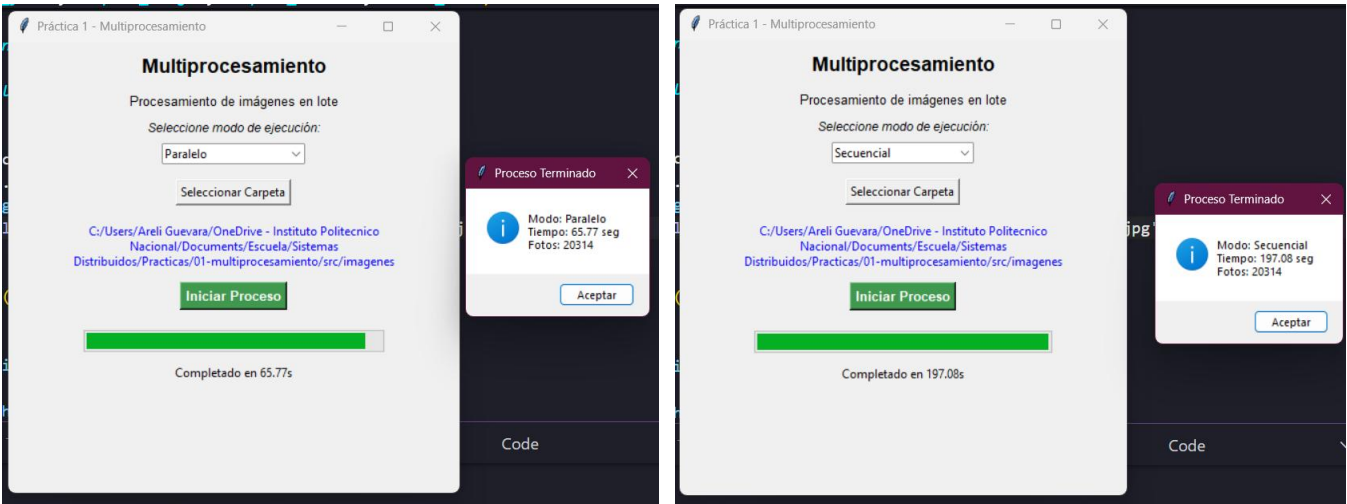
```
1 self.tiempo_inicio = time.time()
2 tiempo_total = time.time() - self.tiempo_inicio
```

Esto permite evaluar el impacto real del multiprocesamiento y comparar experimentalmente los resultados con el modo secuencial, el cual se utilizó únicamente como referencia para demostrar la mejora en rendimiento.

RESULTADOS

Para evaluar el impacto del modelo de multiprocesamiento por hilos, se realizaron pruebas utilizando el mismo conjunto de datos en ambos modos de ejecución. El lote de prueba estuvo compuesto por **20,314 imágenes**, las cuales fueron sometidas al mismo proceso de transformación (conversión a escala de grises).

Las pruebas se ejecutaron en las mismas condiciones de hardware y software, variando únicamente el modo de ejecución.



Los tiempos registrados fueron los siguientes:

Modo de ejecución	Número de imágenes	Tiempo total
Secuencial	20,314	197.08 s
Paralelo	20,314	65.77 s

Práctica 1: Multiprocesamiento

A partir de los resultados, se observa una mejora significativa al utilizar el modelo de multiprocesamiento. El tiempo de ejecución se redujo de **197.08 segundos a 65.77 segundos**, lo que representa una disminución aproximada del:

$$\text{Mejora} = \frac{197.08 - 65.77}{197.08} \times 100 \approx 66.6\%$$

Es decir, el procesamiento paralelo logró completar la misma carga de trabajo en **aproximadamente un tercio del tiempo** requerido por el enfoque secuencial.

La reducción del tiempo se debe a que, en el modo paralelo, las imágenes no se procesan de manera lineal, sino que el lote total se divide entre varios hilos que se ejecutan de forma concurrente. Esto permite aprovechar múltiples núcleos del procesador y distribuir la carga de trabajo.

Mientras que en el modo secuencial cada imagen debe esperar a que la anterior termine completamente, en el modo paralelo varias imágenes son procesadas simultáneamente, eliminando los tiempos muertos y mejorando la eficiencia global del sistema.

CONCLUSIONES

Los resultados obtenidos validan la propuesta de solución planteada, demostrando que el uso de multiprocesamiento mediante hilos es una estrategia efectiva para resolver la ineficiencia del procesamiento masivo de imágenes.

La implementación paralela no solo reduce significativamente los tiempos de espera, sino que también demuestra la importancia de aplicar técnicas de concurrencia en aplicaciones que manejan grandes volúmenes de datos. En este caso particular, el uso de hilos permitió obtener una mejora de rendimiento superior al 60%, evidenciando una optimización clara frente al modelo tradicional secuencial.